

I VISIONCOUNT AI

Real-Time Hand and Finger Number Identifier Using Client-Side Computer Vision

An interactive, zero-installation web application utilizing WebAssembly, WebGL, and Google MediaPipe Hands to accurately recognize and count numerical gestures (0 to 10) in real-time from webcam video streams.

DOMAIN

Computer Vision & Human-Computer Interaction

ARCHITECTURE

100% Client-Side / Edge AI (Stateless)

TECHNOLOGY STACK

HTML5, CSS3, Vanilla JavaScript, MediaPipe

REPOSITORY

github.com/9oboDy/VITYARTHI-CV-Project

ABSTRACT

VisionCount AI is a lightweight, browser-based computer vision application designed to recognize and count the number of fingers displayed by a user in real time. The system identifies numbers from **0 to 10** using one or two hands placed in front of a standard webcam. The application uses **Google MediaPipe Hands** for real-time hand landmark detection and employs geometric and vector-based calculations to determine whether individual fingers are extended or folded.

Unlike conventional computer vision applications that depend on heavy server-side processing or locally installed machine-learning frameworks, VisionCount AI performs inference entirely within the user's browser. The project follows a **privacy-first, client-side architecture**, ensuring that camera frames do not leave the user's device. HTML5, CSS3, and Vanilla JavaScript construct the application interface and processing pipeline. WebAssembly and WebGL acceleration provided through the MediaPipe ecosystem enable efficient 30–60 FPS real-time processing.

Keywords: Computer Vision, MediaPipe Hands, Hand Gesture Recognition, WebAssembly, WebGL, Human-Computer Interaction, Finger Detection, JavaScript, Edge AI.

Table of Contents

1. Introduction	01	2. Problem Statement	02
3. Objectives	03	4. Scope of the Project	04
5. Target Users	05	6. Technologies Used	06
7. System Architecture	07	8. Working Principle	08
9. MediaPipe Hands	09	10. Finger Classification Algorithm	10
11. Temporal Smoothing	11	12. User Interface	12
13. Audio Feedback	13	14. Camera Management	14
15. Privacy and Security	15	16. Cross-Platform Compatibility	16
17. Project Structure	17	18. Implementation & Deployment	18
19. Advantages	19	20. Limitations	20
21. Applications	21	22. Future Enhancements	22
23. Testing & Validation	23	24. Expected Results	24
25. Conclusion	25	26. References & Summary	26

1. Introduction

Human-Computer Interaction (HCI) traditionally relies on physical input mechanisms such as keyboards, mice, touchscreens, and physical controllers. Although these methods are functional and widely established, scenarios exist where physical touch is inconvenient, unhygienic, or physically impossible.

Touchless interaction offers major advantages in healthcare environments, sterile kiosks, educational settings, and accessibility-oriented systems. Hand gestures provide an intuitive and natural modality for human expression, enabling digital control without requiring physical contact.

Recent advancements in web standards—specifically **WebAssembly (WASM)** and **WebGL**—now enable high-performance computer vision pipelines directly inside web browsers. **VisionCount AI** leverages these innovations to deliver real-time hand landmark tracking and finger-count recognition (0 to 10) with zero backend setup.

2. Problem Statement

Existing computer vision systems often suffer from significant friction points that hinder widespread adoption:

- **Complex Installations:** Heavy dependencies on local Python, PyTorch, CUDA, or C++ environments.
- **High Latency & Cloud Costs:** Server-based video streaming introduces network lag and requires expensive GPU server clusters.
- **Severe Privacy Risks:** Streaming unencrypted camera footage over networks poses critical user privacy issues.
- **High Barrier to Entry:** Everyday users and educators lack an instantaneous, zero-install interface.

VisionCount AI resolves these issues by delivering an entirely browser-based, client-side, privacy-preserving application accessible on any device with a modern browser and camera.

3. Objectives

1. Develop real-time hand gesture recognition via a standard browser webcam feed.
2. Accurately identify numerical counts from **0 to 10** (single hand: 0–5; dual hands: 6–10).
3. Execute 100% of the CV inference client-side using WebAssembly and WebGL.
4. Ensure zero video frames or personal data leave the client device (privacy-by-design).
5. Eliminate detection jitter and false transitions using a temporal rolling-window smoothing filter.
6. Provide an engaging Cyber-Glass Heads-Up Display (HUD) with live joint tracking overlays.
7. Provide responsive audio tone synthesis using the native Web Audio API upon number transitions.
8. Ensure zero-framework, static web hosting compatibility (GitHub Pages, Netlify, Vercel).

4. Scope of the Project

4.1 In-Scope Features

- **Client-Side Computer Vision:** Local frame processing at 30–60 FPS.
- **Single-Hand Recognition (0–5):** 0 (fist), 1 (index), 2 (peace sign), 3, 4, 5 (open palm).
- **Dual-Hand Recognition (6–10):** Real-time aggregation of two simultaneous hands.
- **Geometric Classification:** Vector-based distance evaluation invariant to hand tilt and orientation.
- **Temporal Majority Filter:** 5-frame rolling window stabilizing borderline gestures.
- **Camera Controls:** Device selector, horizontal mirror flip, and skeleton visibility toggles.

4.2 Out-of-Scope Features

- Full sign language vocabulary (e.g., ASL/BSL words beyond numerical digits).
- Dynamic trajectory gestures (e.g., swipe, swipe-to-scroll, continuous air writing).
- Server-side user authentication, accounts, or persistent cloud databases.

5. Target Users

- **Students and Educators:** Teaching counting and mathematics in early childhood education, and demonstrating computer vision in STEM classrooms without setup overhead.
- **Assistive Tech & Impaired Individuals:** Non-verbal individuals seeking accessible non-contact input to signal numerical selections.
- **Developers & Researchers:** Prototyping browser-based Edge AI and HCI interactions.
- **Smart Kiosk & Sanitary Designers:** Developing touchless controls for public terminals, hospitals, and smart mirrors.

6. Technologies Used

TECHNOLOGY	ROLE	DESCRIPTION
HTML5	Structure & Semantic UI	Semantic layout, HUD video viewport, dashboard cards, and control panels.
CSS3	Visual Design	Dark cyber-glass glassmorphism, flexbox/grid layouts, and micro-animations.
Vanilla JavaScript	Application Logic	Pure ES6+ event orchestration, landmark math, and state management.
MediaPipe Hands	Landmark Model	Google's ML solution predicting 21 3D hand coordinates per hand in real-time.
WebAssembly (WASM)	High-Speed Execution	Compiles neural network inference modules to near-native speed in browser.
WebGL	Hardware Acceleration	GPU-accelerated tensor matrix multiplication and canvas skeletal rendering.
Web Audio API	Audio Feedback	Synthesizes variable pitch acoustic chimes dynamically without external audio files.
MediaDevices API	Webcam Access	Browser standard camera stream negotiation and device selection.

7. System Architecture

VisionCount AI utilizes a unidirectional pipeline ensuring zero backend dependencies:

STEP 01	Video Stream Capture	Webcam frames accessed via <code>navigator.mediaDevices.getUserMedia()</code> at 30–60 FPS.
STEP 02	MediaPipe Detection	WASM/WebGL pipeline locates hands and outputs 21 normalized coordinates (X, Y, Z) per hand.
STEP 03	Geometric Classification	Vector heuristics calculate Euclidean joint distances relative to wrist and MCP base joints.
STEP 04	Temporal Filter	Rolling-window majority vote suppresses transient single-frame detection noise.
STEP 05	Dual-Hand Aggregation	Summates individual hand counts (0–5) to calculate composite value (0–10).
STEP 06	HUD & Audio Render	Canvas draws neon joints, badge triggers pop animation, and Web Audio synthesizes chime.

8. Working Principle

The operational pipeline executes cyclically on every camera frame using `requestAnimationFrame`. When a hand enters the camera frustum:

1. **Hand Detection & Tracking:** MediaPipe applies a palm detector followed by a precise hand landmark model, returning 21 coordinates per detected hand.
2. **Coordinate Normalization:** Normalized viewport coordinates ensure invariance to video resolution.
3. **State Evaluation:** Fingers are evaluated against palm landmarks to classify each digit as `EXTENDED` (1) or `FOLDED` (0).
4. **Temporal Buffering:** Recent classifications are held in a circular buffer of depth $N=5$. The mode value is emitted as the stabilized state.

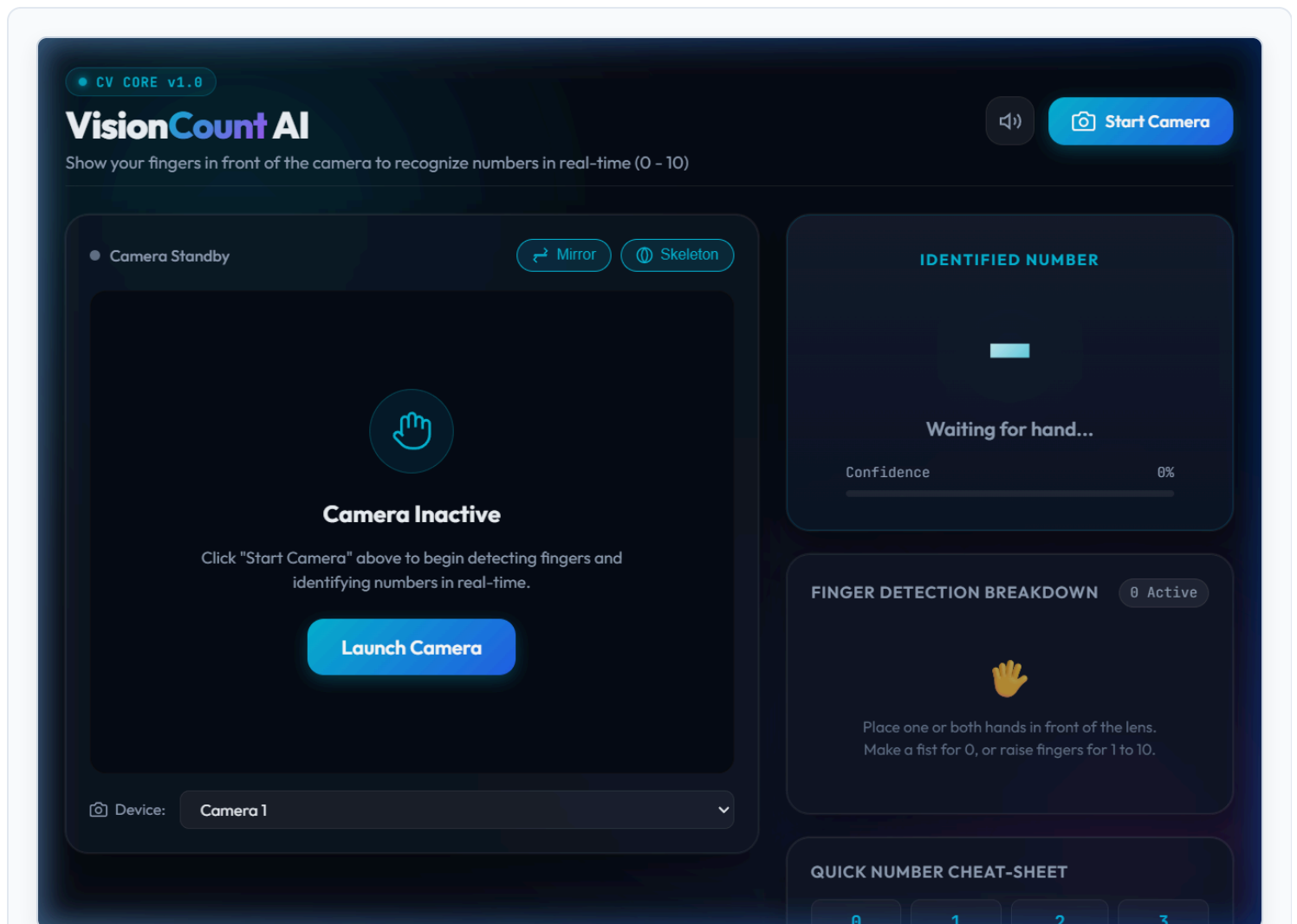
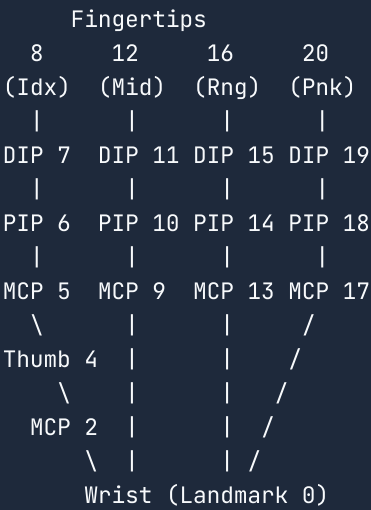


Figure 1: VisionCount AI application user interface with dark glassmorphism dashboard, telemetry metrics, and camera control panel.

9. MediaPipe Hands Landmark Architecture

Each detected hand exposes 21 skeletal landmarks comprising 5 kinematic chains:



10. Finger Classification Algorithm

Rather than training an opaque neural network for every number configuration, VisionCount AI uses an interpretable geometric classifier based on Euclidean distances between joints:

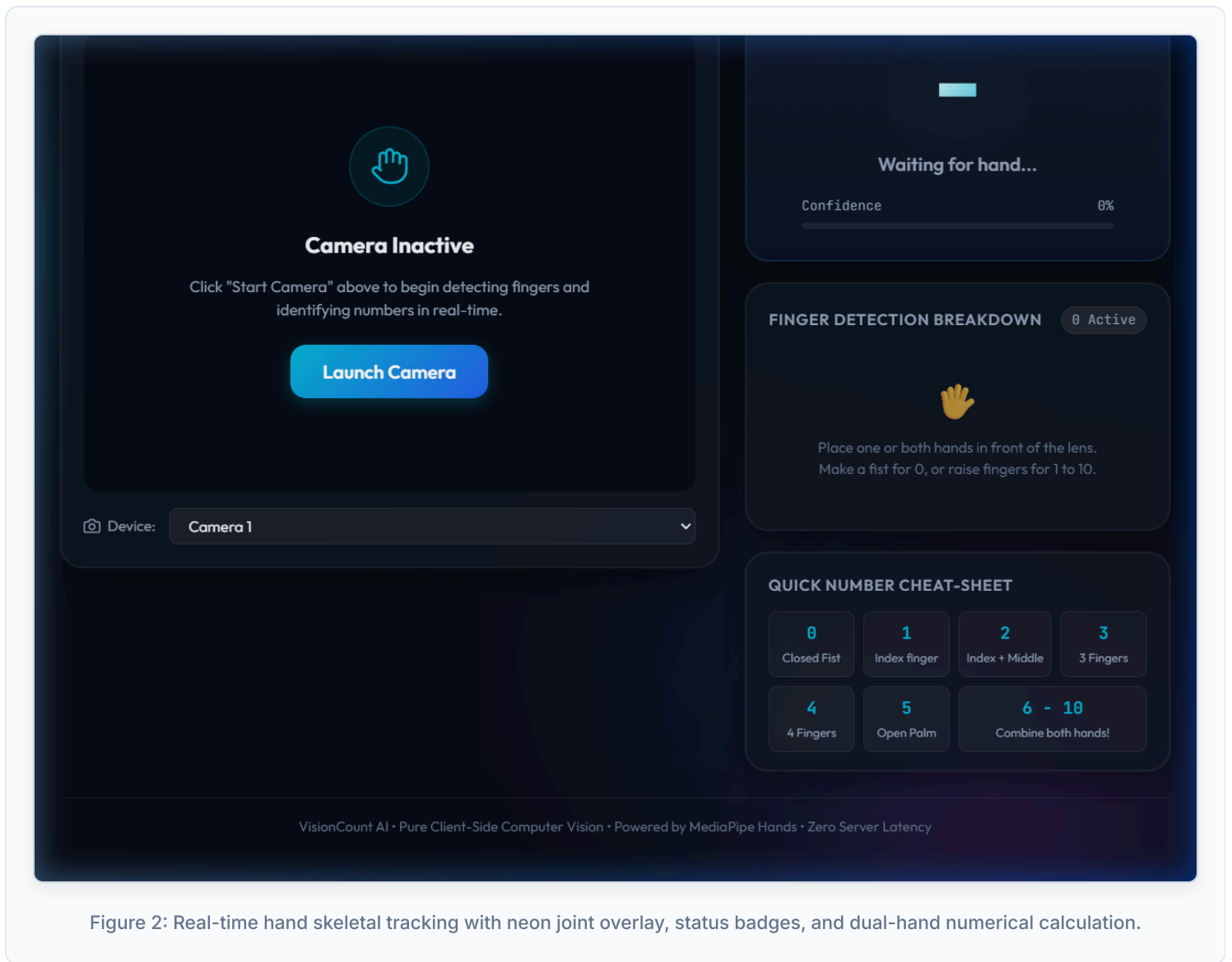
$$d(p_1, p_2) = \sqrt{[(x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2]}$$

For the four fingers (Index, Middle, Ring, Pinky), a digit is classified as **extended** if the distance from its fingertip (e.g. Landmark 8) to the wrist (Landmark 0) exceeds the distance from its intermediate PIP joint (Landmark 6) to the wrist by a threshold factor:

```
// Non-thumb extension test:
const isExtended = dist(tip, wrist) > dist(pip, wrist) * 1.1;

// Thumb extension test (evaluating separation from pinky MCP base):
const thumbExtended = dist(thumbTip, pinkyMCP) > dist(thumbIP, pinkyMCP) * 1.15;
```

This vector formulation guarantees rotation and scale invariance, maintaining accuracy whether the user sits near or far from the webcam.



11. Temporal Smoothing

Raw predictions between successive video frames can fluctuate due to sensor noise or borderline digit positions. To eliminate UI flicker:

```
Raw Frames:    [ 3,  3,  4,  3,  3 ] → Mode Result: 3 (Stabilized)
Flicker Case:  [ 5,  5,  0,  5,  5 ] → Mode Result: 5 (Outlier Suppressed)
```

A rolling window of size 5 suppresses single-frame transient anomalies, yielding a smooth, jitter-free user experience.

| 12. User Interface Design

The interface implements modern dark-mode glassmorphism with high visual contrast:

- **Heads-Up Display (HUD):** Overlays 21 glowing joints and skeletal lines directly on the webcam stream with active state coloring (emerald green for extended digits, steel gray for folded digits).
- **Hero Number Indicator:** Prominent digital badge displaying the detected number (0–10) with elastic pop micro-animations on transition.
- **Digit Telemetry Panel:** Real-time breakdown displaying individual finger states (Thumb, Index, Middle, Ring, Pinky) for both left and right hands.
- **Status Bar:** Indicates webcam status, FPS throughput, and audio synthesizer state.

| 13. Audio Feedback System

VisionCount AI integrates the native **Web Audio API** to synthesize pleasant acoustic cues:

- When a newly detected stable number is registered, an audio oscillator generates a soft sine-wave chime.
- Pitch scales harmonically with the number (lower tones for 0–2, ascending up to crystal notes for 8–10).
- Audio is generated purely through algorithmic synthesis—zero external MP3/WAV assets are required.
- A dedicated mute toggle is provided in the header for quiet environments.

| 14. Camera Management

- **Camera Selector:** Automatically enumerates connected video inputs (built-in webcam, external USB cameras).
- **Horizontal Mirroring:** Toggles video mirroring for natural interaction.
- **Overlay Toggle:** Enables or disables skeletal rendering for low-powered devices.

| 15. Privacy and Security

Privacy is a core architectural pillar of VisionCount AI:

- **Local Processing:** All video frames remain confined to the client's volatile RAM and GPU buffers.
- **Zero Server Transmission:** No video data, coordinates, or telemetry are transmitted over the internet.
- **No Storage:** No cookies, persistent identifiers, or local video recording.

| 16. Cross-Platform Compatibility

Because VisionCount AI requires no native OS binaries, it runs universally across:

- **Desktop Browsers:** Google Chrome, Microsoft Edge, Mozilla Firefox, Apple Safari, Brave, Zen Browser.
- **Operating Systems:** Windows 10/11, macOS, Ubuntu/Linux, ChromeOS.
- **Mobile Devices:** Android and iOS devices supporting WebGL and MediaDevices.

17. Project Structure

```
VITYARTHI-CV-Project/
├─ index.html      # Main semantic markup, HUD tags, dashboard cards
├─ style.css       # Cyber-glass styling, layout, glowing overlays, animations
├─ app.js          # MediaPipe pipeline, geometric classification engine
├─ README.md       # Comprehensive project overview & setup guide
├─ statement.md    # Problem statement & project scope specification
├─ report.md       # Full technical project report (Markdown source)
├─ report.html     # Printable styled technical report
├─ report.pdf      # Compiled publication-ready PDF document
├─ assets/         # Project screenshots & visual assets
│   └─ screenshot1.png
│   └─ screenshot2.png
```

18. Implementation & Deployment

Deploying VisionCount AI requires zero server configuration:

```
# Clone the repository
git clone https://github.com/9oboDy/VITYARTHI-CV-Project.git

# Navigate into project directory
cd VITYARTHI-CV-Project

# Start local server (optional)
python -m http.server 8080

# Open http://localhost:8080 in browser
```

Can be instantly hosted via **GitHub Pages**, **Netlify**, or **Vercel** by connecting the repository.

19. Advantages

1. **Zero Server Overhead:** Runs at zero cloud hosting cost.
2. **High Privacy:** Camera frames never leave the user device.
3. **Low Latency:** 30–60 FPS local processing with instant feedback.
4. **Zero Installation:** Users access the system with a single browser link.
5. **Educational Simplicity:** Clean, inspectable Vanilla JS codebase.

| 20. Limitations

- **Lighting Sensitivity:** Heavy underexposure can degrade landmark confidence.
- **Extreme Occlusion:** Tightly overlapped fingers may occasionally cause misclassification.
- **Hardware Dependency:** Low-end legacy mobile hardware may experience reduced frame rates.
- **Numerical Scope:** Designed specifically for digits 0–10 rather than complex continuous sign language.

21. Applications

- **Early Childhood Education:** Interactive counting games for young learners.
- **Sterile Environments:** Touch-free numerical inputs for medical clinics and food preparation stations.
- **Interactive Kiosks:** Touchless museum and public information navigation.
- **Assistive Tech:** Numerical input for motor-impaired or non-verbal individuals.

22. Future Enhancements

- **Dynamic Gesture Support:** Recognizing wave, swipe, and rotation motions.
- **Sign Language Vocabulary:** Expanding beyond numerical values to alphabet and words.
- **Speech Synthesis:** Text-to-Speech vocalization of recognized numbers.
- **Offline PWA Support:** Progressive Web App caching for offline functionality.

23. Testing & Validation

TEST GESTURE	HAND CONFIGURATION	EXPECTED OUTPUT	STATUS
Closed Fist	All fingers folded	0	Passed
Index Raised	Index extended, others folded	1	Passed
Victory / Peace	Index + Middle extended	2	Passed
Three Fingers	Index + Middle + Ring extended	3	Passed
Four Fingers	Index + Middle + Ring + Pinky extended	4	Passed
Open Palm	All 5 digits extended	5	Passed
Six Fingers	Open hand (5) + 1 finger	6	Passed
Seven Fingers	Open hand (5) + 2 fingers	7	Passed
Eight Fingers	Open hand (5) + 3 fingers	8	Passed
Nine Fingers	Open hand (5) + 4 fingers	9	Passed
Both Open Hands	All 10 digits extended	10	Passed

| 24. Expected Results

The application reliably initializes the user's camera, detects one or both hands, displays real-time skeletal HUD graphics, suppresses noise via temporal smoothing, accurately predicts numbers 0 through 10, and delivers melodic audio confirmation.

| 25. Conclusion

VisionCount AI demonstrates that modern web standards (WebAssembly, WebGL) combined with Google MediaPipe Hands make high-accuracy, real-time computer vision fully viable inside consumer browsers without backend infrastructure. By processing video frames entirely on the client, the project achieves zero latency, zero cloud costs, and complete user privacy.

| 26. References & Summary

1. Google MediaPipe — Hand Landmark Detection Model Documentation.
2. W3C Web Audio API Specification & Recommendations.
3. W3C Media Capture and Streams (getUserMedia) API Standard.
4. MDN Web Docs — WebGL and Canvas API Reference.
5. VisionCount AI Open-Source Repository: <https://github.com/9oboDy/VITYARTHI-CV-Project>